

AUTOMATED WALL PAINTING MACHINE

Arduino Control Program — Source Code Documentation

Chapter 7 — Circuit Connections and Software Programming
Final Year Project | B.E. Mechanical Engineering | Don Bosco College of Engineering, Goa University | 2022–23
Authors: Abdul Aziz Salman Mohammed & Team | Guide: Prof. Sachin Turi | Grade: O (Outstanding)

1. Program Overview

The Arduino Uno R3 serves as the central control unit of the Automated Wall Painting Machine. The program implements a **closed-loop state machine** that controls all machine movements — vertical spray stroke, obstacle detection, trigger actuation, and lateral shift — in a repeating loop. The program was written in C using the Arduino IDE and uploaded to the ATmega328P microcontroller.

The control logic has five distinct states per painting cycle:

- State 1 — Activate spray gun (extend linear actuator to pull trigger)
- State 2 — Move upward while continuously measuring wall distance with HC-SR04 ultrasonic sensor
- State 3 — Obstacle logic: if distance deviates >2cm from baseline → pause spray; resume when clear
- State 4 — Return downward to home position using bottom limit switch
- State 5 — Lateral shift 75mm to the right using timed geared motor run (1000ms)

The setup() function handles one-time calibration: resets vertical and lateral positions to zero using limit switches, measures the baseline wall-to-sensor distance, and waits for the operator start signal.

2. Hardware Configuration & Pin Mapping

The following pin assignments were used, as extracted directly from the project code (Chapter 7, pp. 62–68 of the FYP report):

Arduino Pin	Component	Signal Type	Function
D2	HC-SR04 Ultrasonic	Digital OUTPUT	Trigger pulse to sensor
D3	HC-SR04 Ultrasonic	Digital INPUT	Echo return from sensor
D4	L298N Motor Driver	Digital OUTPUT	PMDC motor UP direction
D5	L298N Motor Driver	Digital OUTPUT	PMDC motor DOWN direction
D6	L298N Motor Driver	Digital OUTPUT	Geared motor LEFT
D7	L298N Motor Driver	Digital OUTPUT	Geared motor RIGHT
D8	Top Limit Switch	Digital INPUT	Vertical upper end-stop
D9	Bottom Limit Switch	Digital INPUT	Vertical lower end-stop
D10	Left Limit Switch	Digital INPUT	Lateral left end-stop
D11	Right Limit Switch	Digital INPUT	Lateral right end-stop
D12	Start Button	Digital INPUT	Operator start signal
A0	5V Relay Module	Analog as OUTPUT	Actuator extend (trigger ON)

A1	5V Relay Module	Analog as OUTPUT	Actuator retract (trigger OFF)
----	-----------------	------------------	--------------------------------

Note on actuator control: The 100N linear actuator is driven through a 5V relay module. Both relay channels are initialised HIGH (relay open = actuator off). Extending the actuator (trigger pull) = actu_in LOW + actu_out HIGH. Retracting = actu_in HIGH + actu_out LOW. Both HIGH = actuator idle/hold.

3. Complete Source Code

The following is the complete Arduino program as implemented in the FYP prototype, extracted from Chapter 7 (pp. 63–68) of the project report. Comments have been expanded for clarity. No logic has been modified from the original implementation.

```
// =====
// Automated Wall Painting Machine — Arduino Control Program
// Design and Fabrication of Automated Wall Painting Machine for Walls
// Don Bosco College of Engineering, Goa University | FYP 2022-23
// Guide: Prof. Sachin Turi | Grade: O (Outstanding)
// =====

// --- Pin definitions ---
int top_switch = 8;      // Top limit switch (vertical upper end-stop)
int dwn_switch = 9;      // Bottom limit switch (vertical lower end-stop)
int lft_switch = 10;     // Left limit switch (lateral left end-stop)
int rit_switch = 11;     // Right limit switch (lateral right end-stop)
int top = 4;             // Motor driver IN1 — drives PMDC motor UP
int dwn = 5;             // Motor driver IN2 — drives PMDC motor DOWN
int lft = 6;             // Motor driver IN3 — drives geared motor LEFT
int rit = 7;             // Motor driver IN4 — drives geared motor RIGHT
int actu_in = A0;        // Relay signal IN — extends linear actuator (trigger ON)
int actu_out = A1;       // Relay signal OUT — retracts linear actuator (trigger OFF)
int startpin = 12;       // Start button — machine waits here until pressed

// --- Ultrasonic sensor pins (HC-SR04) ---
const int trigPin = 2;   // Trigger output to HC-SR04
const int echoPin = 3;   // Echo input from HC-SR04

// --- Variables ---
float duration, distance, distance_0;
int rit_shift_dely = 1000; // Duration (ms) for one lateral shift step (75mm)
int actu_delay = 4000;    // Duration (ms) to hold linear actuator extended/retracted

// =====
// SETUP — runs once on power-on or reset
// =====
void setup()
{
    // Ultrasonic sensor pin modes
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    Serial.begin(9600);

    // Limit switch and control pin modes
    pinMode(top_switch, INPUT);
    pinMode(dwn_switch, INPUT);
    pinMode(lft_switch, INPUT);
    pinMode(rit_switch, INPUT);
    pinMode(startpin, INPUT);
    pinMode(top, OUTPUT);
    pinMode(dwn, OUTPUT);
}
```

```

pinMode(lft,      OUTPUT);
pinMode(rit,      OUTPUT);
pinMode(actu_in,  OUTPUT);
pinMode(actu_out, OUTPUT);

// Initialise actuator in OFF state (both relay channels HIGH = relay open)
digitalWrite(actu_in,  HIGH);
digitalWrite(actu_out, HIGH);

// --- CALIBRATION 1: Reset vertical position to bottom ---
// Move UP until bottom limit switch is triggered
while (digitalRead(dwn_switch) == HIGH) {
    digitalWrite(top, HIGH);
    digitalWrite(dwn, LOW);
}
digitalWrite(top, HIGH);
digitalWrite(dwn, HIGH); // Stop vertical motor

// --- CALIBRATION 2: Reset lateral position to leftmost ---
while (digitalRead(lft_switch) == HIGH) {
    digitalWrite(lft, HIGH);
    digitalWrite(rit, LOW);
}
digitalWrite(lft, LOW);
digitalWrite(rit, LOW); // Stop lateral motor

// --- CALIBRATION 3: Measure baseline wall distance ---
// Fire ultrasonic pulse and record reference distance from wall
digitalWrite(trigPin, LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
distance_0 = (duration * 0.0343) / 2; // Convert to cm
Serial.print("Baseline distance: ");
Serial.println(distance_0);

// --- WAIT: Hold until operator presses start button ---
while (digitalRead(startpin) == HIGH) {
    // Idle - waiting for operator start signal
}
}

// =====
// LOOP - repeats continuously after setup()
// Each iteration paints one vertical strip, then shifts 75mm right
// =====
void loop()
{
    int i = 0; // Obstacle flag: 0 = clear, 1 = obstacle detected

    // --- STEP 1: ACTIVATE SPRAY GUN ---
    // Extend linear actuator to pull spray gun trigger
    digitalWrite(actu_in,  LOW);
    digitalWrite(actu_out, HIGH);
    delay(actu_delay);
    // Hold trigger position - actuator remains extended during upward movement
    digitalWrite(actu_in,  HIGH);
    digitalWrite(actu_out, HIGH);

    // --- STEP 2: MOVE UPWARD while monitoring for obstacles ---
    while (digitalRead(top_switch) == HIGH) {
        delayMicroseconds(500);

        // Fire ultrasonic pulse
        digitalWrite(trigPin, LOW);
    }
}

```

```

    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
    distance = (duration * 0.0343) / 2;
    Serial.print("Distance: ");
    Serial.println(distance);

    // OBSTACLE CLEAR: distance matches baseline → resume spray if previously
    paused
    if ((distance < (distance_0 + 2)) &&
        (distance > (distance_0 - 2)) &&
        (i == 1))
    {
        digitalWrite(actu_in, LOW); // Extend actuator – trigger ON
        digitalWrite(actu_out, HIGH);
        delay(actu_delay);
        digitalWrite(actu_in, HIGH);
        digitalWrite(actu_out, HIGH);
        i = 0; // Clear obstacle flag
        Serial.print("Spray: ON ");
    }
    // OBSTACLE DETECTED: distance outside ±2cm baseline → pause spray
    else if ((distance > (distance_0 + 2)) ||
        (distance < (distance_0 - 2)))
    {
        digitalWrite(actu_in, HIGH); // Retract actuator – trigger OFF
        digitalWrite(actu_out, LOW);
        delay(actu_delay);
        digitalWrite(actu_in, HIGH);
        digitalWrite(actu_out, HIGH);
        i = 1; // Set obstacle flag
        Serial.print("Spray: OFF (obstacle)");
    }

    // Continue moving UP while monitoring
    digitalWrite(top, LOW);
    digitalWrite(dwn, HIGH);
    delayMicroseconds(10);
}

// --- STEP 3: REACHED TOP – stop vertical motor ---
digitalWrite(top, HIGH);
digitalWrite(dwn, HIGH);

// Turn off spray if still active at top
if (i == 0) {
    digitalWrite(actu_in, HIGH);
    digitalWrite(actu_out, LOW);
    delay(actu_delay);
    digitalWrite(actu_in, HIGH);
    digitalWrite(actu_out, HIGH);
}
delay(1000);

// --- STEP 4: RETURN DOWNWARD to home position ---
while (digitalRead(dwn_switch) == HIGH) {
    digitalWrite(top, HIGH);
    digitalWrite(dwn, LOW);
}
digitalWrite(top, HIGH);
digitalWrite(dwn, HIGH); // Stop at bottom

// Ensure actuator is reset
digitalWrite(actu_in, HIGH);
digitalWrite(actu_out, HIGH);

```

```

delay(1000);

// --- STEP 5: LATERAL SHIFT — move 75mm to the right ---
digitalWrite(lft, LOW);
digitalWrite(rit, HIGH);
delay(rit_shift_dely); // 1000ms ≈ 75mm lateral shift

// Stop lateral motor
digitalWrite(lft, LOW);
digitalWrite(rit, LOW);

// Loop repeats — paints next vertical strip
}

```

4. Control Logic Explanation

4.1 Setup and Calibration

On power-on or reset, the `setup()` function executes once. It initialises all pin modes, resets both axes to their zero positions using the limit switches, measures the baseline wall distance (`distance_0`) using the HC-SR04 ultrasonic sensor, and holds at the `while(startpin)` loop until the operator presses the start button. This ensures the machine always begins from a known, calibrated state.

4.2 Obstacle Detection Logic

During the upward stroke, the ultrasonic sensor fires a $10\mu\text{s}$ pulse every $500\mu\text{s}$ loop cycle. The returned echo duration is converted to distance in cm using the formula: $\text{distance} = (\text{duration} \times 0.0343) / 2$. If the measured distance deviates by more than $\pm 2\text{cm}$ from the baseline (`distance_0`), an obstacle is detected — the relay retracts the actuator, pausing the spray. When distance returns within $\pm 2\text{cm}$ and the obstacle flag (`i=1`) is set, the actuator extends again and the flag resets to 0.

4.3 Lateral Shift

After each complete vertical stroke (up and back down), the geared box motor runs RIGHT for 1000ms (`rit_shift_dely`). This was calibrated during testing to correspond to approximately 75mm of lateral movement — the strip width for each pass. The `loop()` function then repeats, painting the next vertical strip.

4.4 Known Limitations of This Implementation

- No wall boundary detection — the machine does not detect when it reaches the right wall edge; an operator must stop the machine manually
- Fixed lateral shift timing — 1000ms is approximate; actual shift distance varies slightly with battery voltage and motor temperature
- No multi-strip counter — the program does not track how many strips have been painted
- Obstacle detection threshold is fixed at $\pm 2\text{cm}$ — this was tuned empirically during testing
- The `if(i==0)` at line 81 of the original source contains a capitalisation error (should be `if`) — functionally equivalent in Arduino IDE but flagged here for accuracy

5. Key Variables Reference

Variable	Type	Value	Purpose
<code>distance_0</code>	float	Measured at startup	Baseline wall distance from sensor at calibrated position
<code>distance</code>	float	Updated each loop	Current wall distance reading during spray stroke
<code>duration</code>	float	Updated each loop	Raw HC-SR04 echo pulse duration in microseconds

i	int	0 or 1	Obstacle flag: 0=clear, 1=obstacle detected
rit_shift_dely	int	1000 ms	Time geared motor runs right per lateral shift
actu_delay	int	4000 ms	Time linear actuator holds extended or retracted

Abdul Aziz Salman Mohammed & Team | B.E. Mechanical Engineering | Don Bosco College of Engineering, Goa University | 2022–23 | github.com/AbdulAzizSalman23